

Debugging Commands in Bash (Terminal Style)

Definition:

Debugging in Bash helps identify and fix errors by tracing, printing, and controlling script execution.

Common Debugging Options

Option	Description
----- -----	
set -x	Print each command before executing
set +x	Stop printing commands
bash -x script.sh	Run script in debug mode
set -e	Exit immediately if command fails
set -v	Print lines before executing
trap ERR	Run command when an error occurs
echo	Print variable values manually

Example 1: Using set -x and set +x

```
#!/bin/bash
echo "Start script"
set -x
ls /home
pwd
set +x
echo "End script"
```

Example 2: Running script with bash -x

```
bash -x myscript.sh
```

Example 3: set -e (Exit on Error)

```
#!/bin/bash
set -e
echo "Start"
ls /no_dir      # This fails
echo "End"      # Will not execute
```

Example 4: set -u (Unset Variables)

```
#!/bin/bash
set -u
echo $name
```

Example 5: Using trap to Handle Errors

```
#!/bin/bash
trap 'echo "Error on line $LINENO"' ERR
echo "Running command..."
ls /no_such_dir
echo "This will not run"
```

Example 6: Customize Debug Output with PS4

```
PS4='+ [{BASH_SOURCE}:${LINENO}] '
set -x
echo "Testing script"
ls /etc
```

Example 7: Syntax Check Only (-n)

```
bash -n script.sh
```

Summary Table

Option	Description	Example
-----	-----	-----
set -x	Print each command	set -x
set +x	Stop debug printing	set +x
bash -x	Run script in debug mode	bash -x script.sh
set -e	Exit on first error	set -e
set -u	Error on unset variable	set -u
trap ERR	Run command on error	trap 'cmd' ERR
PS4	Customize debug prefix	PS4='+ [\${LINENO}] '
bash -n	Syntax check only	bash -n script.sh